# CASE STUDY 1 – PROGRESSIVE ELECTRICITY BILLING

## 1. Problem Statement

Develop a Python program to calculate household electricity bills using consumption slabs, fixed charges, surcharges, and rebates. Compare monthly bills and identify an unusual increase.

## 2. Objectives

- Calculate charges based on electricity consumption.
- Apply fixed charges, surcharge, and rebate.
- Compare bills across months.
- Identify a significant increase in the bill.

## 3. Assumptions

- First 100 units: ₹2 per unit.
- Next 100 units: ₹3 per unit.
- Above 200 units: ₹5 per unit.
- Fixed charge: ₹50.
- Surcharge: 5% of energy charges.
- Rebate: ₹20 when consumption is below 100 units.
- An increase above 30% is considered unusual.

## 4. Algorithm

1. Store monthly consumption in a list.
2. Read the number of months and units consumed.
3. Calculate slab-wise energy charges.
4. Add fixed charges and surcharge.
5. Apply the rebate when eligible.
6. Compare consecutive bills.
7. Display bills and unusual increases.

## 5. Flowchart

START
↓
Input monthly consumption
↓
Calculate slab-wise charges
↓
Add fixed charge and surcharge
↓
Apply eligible rebate
↓
Store monthly bill
↓
Compare consecutive bills
↓
Increase above 30%?
↓
YES → Report unusual increase
NO → Report normal increase
↓
Display all bills
↓
STOP

# 6. Pseudocode

START
Read number of months
FOR each month
  Read units
  Calculate energy charges using slabs
  Add fixed charge and surcharge
  Apply rebate if eligible
  Store bill
  Compare with previous bill
END FOR
Display monthly bills and unusual increases
STOP

# 7. Python Code

```
months = []
rates = (2, 3, 5)

n = int(input("Enter number of months: "))
```

```
for i in range(n):
    month = input("Enter month name: ")
    units = int(input("Enter units consumed: "))

    if units < 0:
        print("Invalid units")
        continue

    if units <= 100:
        energy = units * rates[0]
    elif units <= 200:
        energy = 100 * rates[0] + (units - 100) * rates[1]
    else:
        energy = 100 * rates[0] + 100 * rates[1] + (units - 200) * rates[2]

    bill = energy + 50 + energy * 0.05

    if units < 100:
        bill = bill - 20

    months.append((month, units, round(bill, 2)))

for i in range(len(months)):
    print(months[i])

    if i > 0:
        previous = months[i - 1][2]
        current = months[i][2]

        if current > previous * 1.30:
            print("Unusual increase detected")
```

# 8. Test Cases

| Units | Expected result |
|---|---|

| | |
|---|---|
| 80 | Bill ₹198.00 |
| 150 | Bill ₹365.00 |
| 250 | Bill ₹785.00 |
| -10 | Invalid units |

## 9. Output Screenshots

Sample console output:

```
Enter number of months: 2
Enter month name: June
Enter units consumed: 100
Enter month name: July
Enter units consumed: 150
('June', 100, 240.0)
('July', 150, 365.0)
Unusual increase detected
```

## 10. Conclusion

The program calculates electricity bills using progressive slabs and additional charges. It also compares monthly bills and identifies unusual increases using lists, tuples, loops, and conditional statements.

Correction for Case Study 1: With the stated slab rates and charges, the correct bills are ₹198 for 80 units, ₹417.50 for 150 units, and ₹837.50 for 250 units. For 100 units, the bill is ₹240.

# CASE STUDY 2 – HOSPITAL BILLING AND PATIENT CLASSIFICATION

# 1. Problem Statement

Create a Python program to calculate hospital bills from consultation, diagnostic, and treatment charges. Apply concessions and classify patients according to the final amount.

# 2. Objectives

- Calculate total hospital charges.
- Apply eligible concessions.
- Validate charges and classify patients.
- Display each patient's final bill.

# 3. Assumptions

Consultation fee = ₹500, diagnostic fee = ₹1,000, and treatment charges are entered by the user. Senior citizens receive a 10% concession. Bills above ₹5,000 are classified as high-cost cases.

# 4. Algorithm

1. Store patient details in a list of tuples.
2. Calculate total charges.
3. Apply the senior citizen concession.
4. Validate the final amount.
5. Classify and display each bill.

# 5. Flowchart

START → Input patient details → Calculate total charges
↓
Senior citizen?
├─ YES → Apply 10% concession
└─ NO → Keep original amount
↓
Validate bill → Classify patient → Display bill → STOP

# 6. Pseudocode

START

Read patient details

FOR each patient

Calculate total charges

IF senior citizen

Apply concession

END IF

Validate and classify bill

Display final amount

END FOR

STOP

# 7. Python Code

```
patients = [
    ("Anu", 60, 2000),
    ("Ravi", 30, 4500),
    ("Sita", 70, 500)
]

for name, age, treatment in patients:
    consultation = 500
    diagnosis = 1000

    if treatment < 0:
        print(name, "Invalid treatment charge")
        continue

    total = consultation + diagnosis + treatment

    if age >= 60:
        total = total - total * 0.10

    if total > 5000:
        category = "High-cost"
    else:
        category = "Normal"

    print(name, round(total, 2), category)
```

## 8. Test Cases

| Patient | Age | Treatment | Expected result |
|---|---|---|---|
| Anu | 60 | ₹2,000 | ₹3,150, Normal |
| Ravi | 30 | ₹4,500 | ₹6,000, High-cost |
| Sita | 70 | ₹500 | ₹1,800, Normal |

## 9. Output Screenshots

Anu 3150.0 Normal

Ravi 6000 High-cost

Sita 1800.0 Normal

## 10. Conclusion

The program calculates patient bills, applies age-based concessions, and classifies patients using conditional statements, lists, tuples, and loops.

# CASE STUDY 3 – RAILWAY FARE AND RESERVATION CALCULATOR

## 1. Problem Statement

Develop a program to calculate railway fares for passengers using distance, travel class, age concessions, reservation fees, and cancellation penalties.

## 2. Objectives

- Calculate distance-based fares.
- Apply class-wise fare rates.
- Provide age-based concessions.
- Calculate reservation fees and cancellation charges.

# 3. Assumptions

- General class: ₹1 per km.
- AC class: ₹2 per km.
- Reservation fee: ₹40.
- Children below 5 travel free.
- Passengers aged 60 or above receive a 20% fare concession.
- Cancellation penalty: 25% of the fare before reservation fees.

# 4. Algorithm

1. Store passenger details in tuples.
2. Calculate fare using distance and class.
3. Apply age concession.
4. Add reservation fees.
5. Apply cancellation penalty if required.
6. Display the final amount.

# 5. Flowchart

START → Input passenger details → Select travel class
↓
Calculate distance fare → Check age concession
↓
Add reservation fee → Ticket cancelled?
├─ YES → Calculate cancellation penalty
└─ NO → Display booking fare
↓
Display final amount → STOP

# 6. Pseudocode

START

FOR each passenger

Calculate fare based on class

Apply age concession

Add reservation fee

IF cancelled

Calculate cancellation penalty

END IF

Display amount

END FOR

STOP

# 7. Python Code

```
passengers = [
    ("Arun", 30, 200, "General", "No"),
    ("Meena", 65, 300, "AC", "No"),
    ("Raju", 4, 150, "General", "No"),
    ("Kiran", 25, 100, "General", "Yes")
]

for name, age, distance, travel_class, cancelled in passengers:
    if distance < 0:
        print(name, "Invalid distance")
        continue

    if age < 5:
        fare = 0
    else:
        if travel_class == "General":
            fare = distance * 1
        elif travel_class == "AC":
            fare = distance * 2
        else:
            print(name, "Invalid class")
            continue

        if age >= 60:
            fare = fare - fare * 0.20

    if cancelled == "Yes":
```

```
    amount = fare * 0.25
elif fare == 0:
    amount = 0
else:
    amount = fare + 40

print(name, "Amount:", round(amount, 2))
```

## 8. Test Cases

| Passenger | Details | Expected amount |
|---|---|---|
| Arun | General, 200 km | ₹240 |
| Meena | AC, 300 km, age 65 | ₹520 |
| Raju | Age 4 | ₹0 |
| Kiran | General, 100 km, cancelled | ₹25 |

## 9. Output Screenshots

Arun Amount: 240

Meena Amount: 520.0

Raju Amount: 0

Kiran Amount: 25.0

## 10. Conclusion

The program calculates railway fares and cancellation penalties using tuples, conditional statements, arithmetic operators, and loops.

# CASE STUDY 4 – EXAMINATION RESULT AND GRACE-MARK VALIDATION

## 1. Problem Statement

Create a Python program to process marks in multiple subjects, apply subject-wise pass criteria and grace marks, and generate final results without exceeding maximum marks.

## 2. Objectives

- Calculate total and percentage.
- Check subject-wise pass marks.
- Apply grace marks within the permitted limit.
- Classify students as pass or fail.

## 3. Assumptions

There are three subjects. Each subject has a maximum of 100 marks and a pass mark of 35. A maximum of 5 grace marks may be added to a subject scoring between 30 and 34. Final marks cannot exceed 100.

## 4. Algorithm

1. Store student marks in a list of tuples.
2. Check each subject's marks.
3. Apply permitted grace marks.
4. Check the final subject marks.
5. Calculate total and percentage.
6. Display the result.

## 5. Flowchart

START → Read student marks → Validate marks
↓
Check each subject → Marks between 30 and 34?
 ├─ YES → Add required grace marks
 └─ NO → Keep original marks
↓
Limit marks to 100 → Check all pass criteria
↓
PASS / FAIL → Calculate percentage → Display → STOP

# 6. Pseudocode

START

FOR each student

  Validate marks

  FOR each subject

    Apply permitted grace marks

    Ensure marks do not exceed 100

  END FOR

  Check all subjects

  Calculate percentage

  Display result

END FOR

STOP

# 7. Python Cod

```
students = [
  ("Anu", 60, 70, 80),
  ("Ravi", 32, 65, 70),
  ("Sita", 28, 75, 60)
]

for name, m1, m2, m3 in students:
```

```
marks = [m1, m2, m3]
valid = True

for i in range(len(marks)):
    if marks[i] < 0 or marks[i] > 100:
        valid = False

if not valid:
    print(name, "Invalid marks")
    continue

for i in range(len(marks)):
    if 30 <= marks[i] < 35:
        marks[i] = marks[i] + 5

    if marks[i] > 100:
        marks[i] = 100

total = sum(marks)
percentage = total / 3

if marks[0] >= 35 and marks[1] >= 35 and marks[2] >= 35:
    result = "PASS"
else:
    result = "FAIL"

print(name, marks, total, round(percentage, 2), result)
```

# 8. Test Cases

| Student | Original marks | Expected result |
|---|---|---|
| Anu | 60, 70, 80 | PASS, 70% |
| Ravi | 32, 65, 70 | PASS after grace |

| | | |
|---|---|---|
| Sita | 28, 75, 60 | FAIL |

## 9. Output Screenshots

Anu [60, 70, 80] 210 70.0 PASS

Ravi [37, 65, 70] 172 57.33 PASS

Sita [28, 75, 60] 163 54.33 FAIL

## 10. Conclusion

The program evaluates student results, applies grace marks within the specified policy, and prevents marks from exceeding the maximum.

# CASE STUDY 5 – MULTI-DAY WATER TANK SIMULATION

## 1. Problem Statement

Develop a Python program to simulate a water tank over several intervals by calculating inflow, outflow, shortages, overflow, and threshold conditions.

## 2. Objectives

- Update the tank's water level.
- Detect overflow and shortage conditions.
- Maintain water levels within tank capacity.
- Find the first interval reaching a specified threshold.

## 3. Assumptions

Tank capacity is 1,000 litres. Initial water level is 500 litres. The threshold is 900 litres. Inflow and outflow are entered for each interval.

## 4. Algorithm

1. Set the initial water level and capacity.
2. Read inflow and outflow values.
3. Calculate the new water level.
4. Detect overflow and shortages.
5. Keep the water level within valid limits.
6. Record the first interval reaching the threshold.

# 5. Flowchart

START → Set initial level and capacity → Read inflow/outflow
↓
Calculate new level → Level above capacity?
├─ YES → Record overflow → Set level to capacity
└─ NO → Continue
↓
Level below zero? → Record shortage → Set level to zero
↓
Threshold reached for first time?
├─ YES → Record interval
└─ NO → Continue
↓
Display results → STOP

# 6. Pseudocode

START

Set capacity and initial level

FOR each interval

    Read inflow and outflow

    Calculate new level

    Detect and correct overflow

    Detect and correct shortage

    Check threshold

    Display interval result

END FOR

STOP

# 7. Python Code

```
capacity = 1000
level = 500
threshold = 900
first_interval = 0

inflows = [300, 400, 100, 50]
outflows = [100, 100, 700, 200]

for i in range(len(inflows)):
    level = level + inflows[i] - outflows[i]

    if level > capacity:
        print("Overflow at interval", i + 1)
        level = capacity

    elif level < 0:
        print("Shortage at interval", i + 1)
        level = 0

    if level >= threshold and first_interval == 0:
        first_interval = i + 1

    print("Interval", i + 1, "Level:", level)

if first_interval != 0:
    print("Threshold first reached:", first_interval)
else:
    print("Threshold not reached")
```

# 8. Test Cases

| Interval | Inflow | Outflow | Expected level |
|---|---|---|---|
| 1 | 300 | 100 | 700 |

| | | | |
|---|---|---|---|
| 2 | 400 | 100 | 1,000 after overflow |
| 3 | 100 | 700 | 400 |
| 4 | 50 | 200 | 250 |

## 9. Output Screenshots

Interval 1 Level: 700

Overflow at interval 2

Interval 2 Level: 1000

Interval 3 Level: 400

Interval 4 Level: 250

Threshold first reached: 2

## 10. Conclusion

The program simulates changing water levels and identifies overflow, shortage, and threshold conditions using lists, loops, and conditional statements.

# CASE STUDY 6 – STUDENT TIMETABLE CONFLICT DETECTION

## 1. Problem Statement

Develop a Python program to identify overlapping class periods, detect unavailable rooms, and validate a proposed student timetable using strings, lists, and tuples.

## 2. Objectives

- Store course schedules.
- Detect overlapping periods.
- Identify room conflicts.
- Validate the proposed timetable.

# 3. Assumptions

Each class has a course name, day, start period, end period, and room number. Periods are represented by integers. Two classes conflict if they occur on the same day and their periods overlap.

# 4. Algorithm

1. Store schedules as tuples.
2. Compare every pair of classes.
3. Check the day and period overlap.
4. Check whether the same room is assigned.
5. Display conflicts and validation status.

# 5. Flowchart

START → Store timetable → Select two classes
↓
Same day and overlapping periods?
├─ YES → Check room conflict
└─ NO → Compare next pair
↓
Report conflicts → Validate timetable → STOP

# 6. Pseudocode

START

Store timetable

FOR each pair of classes

    Check same day

    Check period overlap

    IF overlapping

```
        Report schedule conflict

        IF same room

            Report room conflict

        END IF

    END IF

END FOR

Display validation result

STOP
```

# 7. Python Code

```
schedule = [
    ("Python", "Monday", 1, 3, "R101"),
    ("Maths", "Monday", 2, 4, "R101"),
    ("Physics", "Tuesday", 1, 2, "R102")
]

conflict = False

for i in range(len(schedule)):
    for j in range(i + 1, len(schedule)):
        c1 = schedule[i]
        c2 = schedule[j]

        if c1[1] == c2[1]:
            if c1[2] < c2[3] and c2[2] < c1[3]:
                print(c1[0], "conflicts with", c2[0])
                conflict = True

                if c1[4] == c2[4]:
                    print("Room conflict:", c1[4])

if not conflict:
    print("Timetable is valid")
else:
    print("Timetable needs correction")
```

## 8. Test Cases

| Classes | Expected result |
|---|---|
| Python and Maths | Time and room conflict |
| Python and Physics | No conflict |
| Maths and Physics | No conflict |

## 9. Output Screenshots

Python conflicts with Maths

Room conflict: R101

Timetable needs correction

## 10. Conclusion

The program identifies timetable and room conflicts by comparing strings and tuple values using nested loops and conditional statements.

# CASE STUDY 7 – ADVANCED NUMERIC PATTERN GENERATOR

## 1. Problem Statement

Create a Python program to generate a hollow diamond pattern using numbers. The program must validate the given dimension and display an aligned pattern.

## 2. Objectives

- Accept the required pattern size.
- Validate positive dimensions.
- Generate a symmetric hollow diamond.
- Display aligned numeric output.

# 3. Assumptions

The dimension must be a positive integer. The program generates a diamond using consecutive row numbers, with spaces inside the diamond.

# 4. Algorithm

1. Read the number of rows.
2. Validate the input.
3. Generate the upper half of the diamond.
4. Generate the lower half.
5. Print numbers at the boundaries and spaces inside.

# 5. Flowchart

START → Read number of rows → Rows greater than zero?
 ├─ NO → Display invalid dimension → STOP
 └─ YES → Generate upper half
↓
Generate lower half
↓
Display diamond
↓
STOP

# 6. Pseudocode

```
START

Read n

IF n <= 0

    Display invalid dimension

ELSE

    FOR each row in upper half
```

```
        Print boundary numbers and inner spaces

    END FOR

    FOR each row in lower half

        Print boundary numbers and inner spaces

    END FOR

END IF

STOP
```

# 7. Python Code

```
n = int(input("Enter number of rows: "))

if n <= 0:
    print("Invalid dimension")
else:
    for i in range(1, n + 1):
        print(" " * (n - i), end="")
        print(i, end="")

        if i > 1:
            print(" " * (2 * i - 3), end="")
            print(i, end="")

        print()

    for i in range(n - 1, 0, -1):
        print(" " * (n - i), end="")
        print(i, end="")

        if i > 1:
            print(" " * (2 * i - 3), end="")
            print(i, end="")

        print()
```

# 8. Test Cases

| Input | Expected result |
|---|---|
| 1 | Single number |
| 3 | Hollow diamond with 5 rows |
| 0 | Invalid dimension |
| -2 | Invalid dimension |

## 9. Output Screenshots

For input 3:

```
 1
 2 2
3   3
 2 2
  1
```

## 10. Conclusion

The program generates a hollow diamond using nested output operations, loops, and conditional statements. It also validates the input dimension.

# CASE STUDY 8 – STUDENT EMAIL AND IDENTIFIER VALIDATION

# 1. Problem Statement

Develop a Python program to validate student email addresses and registration identifiers according to basic formatting rules. Report missing components and malformed separators.

# 2. Objectives

- Check the structure of email addresses.
- Validate registration identifiers.
- Detect missing components.
- Display valid or invalid results.

# 3. Assumptions

The email must contain exactly one `@`, a non-empty username, and a domain containing a dot. The registration identifier must start with `STU`, followed by a hyphen and four digits.

# 4. Algorithm

1. Read the email and registration ID.
2. Split the email using `@`.
3. Check the username and domain.
4. Validate the registration ID using string operations.
5. Display validation results.

# 5. Flowchart

START → Read email and registration ID
↓
Check email separator and components
↓
Check domain format
↓
Check registration ID prefix and digits
↓
Display validation results
↓
STOP

# 6. Pseudocode

START

Read email and registration ID

Check exactly one @

Check username and domain

Check domain contains a dot

Check ID prefix, separator, and digits

Display valid or invalid messages

STOP

# 7. Python Code

```
email = input("Enter email: ")
reg_id = input("Enter registration ID: ")

parts = email.split("@")
email_valid = False

if len(parts) == 2:
    username, domain = parts

    if username != "" and "." in domain:
        if domain[0] != "." and domain[-1] != ".":
            email_valid = True

if email_valid:
    print("Email: Valid")
else:
    print("Email: Invalid format or missing component")

id_valid = False

if len(reg_id) == 8:
    if reg_id[:3] == "STU" and reg_id[3] == "-":
        if reg_id[4:].isdigit():
            id_valid = True

if id_valid:
    print("Registration ID: Valid")
```

```
else:
    print("Registration ID: Invalid format")
```

## 8. Test Cases

| Email | Registration ID | Expected result |
|---|---|---|
| `anu@gmail.com` | `STU-1234` | Both valid |
| `anu.gmail.com` | `STU-1234` | Email invalid |
| `anu@@gmail.com` | `STU-12AB` | Both invalid |
| `@gmail.com` | `ABC-1234` | Both invalid |

## 9. Output Screenshots

Enter email: anu@gmail.com

Enter registration ID: STU-1234

Email: Valid

Registration ID: Valid

## 10. Conclusion

The program checks email and registration formats using strings, splitting, indexing, and conditional statements. It performs basic format validation, not full verification of an email account.

# CASE STUDY 9 – HACKATHON TEAM FORMATION AND RANKING

## 1. Problem Statement

Create a Python program to validate hackathon team sizes, calculate round-wise scores, and rank teams using predefined tie-breaking rules.

## 2. Objectives

- Validate team sizes.
- Calculate total team scores.
- Rank teams by total score.
- Resolve ties using round-wise scores.

## 3. Assumptions

Each team must contain 2–4 students. There are three scoring rounds. Teams are ranked by total score, then by the Round 3 score, and finally alphabetically.

## 4. Algorithm

1. Store team details in tuples.
2. Validate each team size.
3. Calculate the total score.
4. Store valid teams in a list.
5. Sort teams using nested loops and tie-breaking rules.
6. Display the ranking.

## 5. Flowchart

START → Read team details → Validate team size
↓
Valid team?
├─ NO → Reject team
└─ YES → Calculate total score
↓
Store valid team
↓

Compare team scores
↓
Apply tie-breaking rules
↓
Display ranking → STOP

# 6. Pseudocode

START

FOR each team

Validate team size

IF valid

Calculate total score

Store team and scores

END IF

END FOR

Compare each pair of valid teams

Swap according to ranking rules

Display ranked teams

STOP

# 7. Python Code

```
teams = [
    ("Alpha", 3, 80, 90, 85),
    ("Beta", 4, 90, 80, 85),
    ("Gamma", 2, 85, 85, 90),
    ("Delta", 5, 95, 90, 95)
]

valid_teams = []

for name, size, r1, r2, r3 in teams:
    if size < 2 or size > 4:
```

```
        print(name, "Invalid team size")
        continue

    total = r1 + r2 + r3
    valid_teams.append((name, size, r1, r2, r3, total))

for i in range(len(valid_teams)):
    for j in range(i + 1, len(valid_teams)):
        a = valid_teams[i]
        b = valid_teams[j]

        if (a[5] < b[5] or
            (a[5] == b[5] and a[4] < b[4]) or
            (a[5] == b[5] and a[4] == b[4]
             and a[0] > b[0])):
            valid_teams[i], valid_teams[j] = b, a

for i in range(len(valid_teams)):
    print("Rank:", i + 1, valid_teams[i][0],
          "Score:", valid_teams[i][5])
```

# 8. Test Cases

| Team | Size | Scores | Expected result |
|---|---|---|---|
| Alpha | 3 | 80, 90, 85 | 255 |
| Beta | 4 | 90, 80, 85 | 255 |
| Gamma | 2 | 85, 85, 90 | 260 |

| | | | |
|---|---|---|---|
| Delta | 5 | 95, 90, 95 | Rejected |

## 9. Output Screenshots

Delta Invalid team size

Rank: 1 Gamma Score: 260

Rank: 2 Alpha Score: 255

Rank: 3 Beta Score: 255

## 10. Conclusion

The program validates team composition and ranks teams based on total and round-wise scores. It demonstrates lists, tuples, nested loops, and tie-breaking conditions.

# CASE STUDY 10 – FOOD WASTE MONITORING SYSTEM

## 1. Problem Statement

Develop a Python program to compare meals prepared, meals served, and leftovers across multiple days. Calculate food wastage percentages and identify days exceeding the acceptable limit.

## 2. Objectives

- Record daily food quantities.
- Calculate leftover meals.
- Calculate wastage percentages.
- Identify days exceeding the wastage limit.

## 3. Assumptions

Food wastage is calculated as leftovers divided by meals prepared, multiplied by 100. A wastage percentage above 10% is considered unacceptable. Meals served cannot exceed meals prepared.

# 4. Algorithm

1. Store daily meal data in tuples.
2. Validate the quantities.
3. Calculate leftovers.
4. Calculate the wastage percentage.
5. Compare it with the 10% limit.
6. Display daily results.

# 5. Flowchart

START → Read daily meal data → Validate quantities
↓
Calculate leftovers → Calculate wastage percentage
↓
Wastage above 10%?
├─ YES → Mark excessive wastage
└─ NO → Mark acceptable wastage
↓
Display daily report → STOP

# 6. Pseudocode

```
START

FOR each day

    Read prepared and served meals

    Validate quantities

    Calculate leftovers

    Calculate wastage percentage

    IF percentage > 10

        Report excessive wastage

    ELSE
```

```
        Report acceptable wastage

    END IF

END FOR

STOP
```

# 7. Python Code

```
days = [
    ("Monday", 200, 180),
    ("Tuesday", 250, 220),
    ("Wednesday", 300, 285),
    ("Thursday", 150, 120)
]

for day, prepared, served in days:
    if prepared < 0 or served < 0 or served > prepared:
        print(day, "Invalid meal data")
        continue

    leftovers = prepared - served

    if prepared == 0:
        percentage = 0
    else:
        percentage = leftovers / prepared * 100

    if percentage > 10:
        status = "Excessive wastage"
    else:
        status = "Acceptable wastage"

    print(day, "Leftovers:", leftovers,
          "Wastage:", round(percentage, 2), "%",
          status)
```

# 8. Test Cases

| Day | Prepared | Served | Wastage | Expected result |
|---|---|---|---|---|
| Monday | 200 | 180 | 10% | Acceptable |
| Tuesday | 250 | 220 | 12% | Excessive |
| Wednesday | 300 | 285 | 5% | Acceptable |
| Thursday | 150 | 120 | 20% | Excessive |

# 9. Output Screenshots

Monday Leftovers: 20 Wastage: 10.0 % Acceptable wastage

Tuesday Leftovers: 30 Wastage: 12.0 % Excessive wastage

Wednesday Leftovers: 15 Wastage: 5.0 % Acceptable wastage

Thursday Leftovers: 30 Wastage: 20.0 % Excessive wastage

# 10. Conclusion

The program monitors daily food wastage and identifies days exceeding the acceptable limit. It demonstrates how lists, tuples, arithmetic operations, loops, and conditional statements can solve a practical problem.